

Erasure Codes for Distributed Storage: Implementation and Trace-Driven Evaluation

Xuanlin Jiang
Harvard University CS 2640

Abstract

Erasure codes promise fault tolerance with less storage than replication, but their I/O latency under realistic workloads — particularly while a disk is out — is poorly characterised. The gap is sharpest for *regenerating* codes (MSR, MBR): despite over a decade of theoretical work establishing their optimality on the storage–repair–bandwidth tradeoff, no mainstream erasure-coding library ships them, and the academic literature on them is dominated by code constructions and information-theoretic bounds rather than empirical benchmarks against the RAID/RS family they aim to displace. I implement four representative codes (RAID-10, Reed–Solomon, MSR, MBR) under a single C++17 interface and drive them with a trace-replay discrete-event simulator over Microsoft Research Cambridge block I/O traces, yielding a side-by-side comparison on the same workload, simulator and failure scenario. The central abstraction is a *DiskRequest DAG* with `on_ready` closures, which lets each code express read–compute–write dependencies (parity deltas, regenerating-code repair matrices) without coupling its math to the simulator. I reproduce the textbook storage–repair tradeoff at fixed (n, k) and find that the degraded-mode latency penalty of regenerating codes is a fixed α -symbol cost that amortises with dataset size: at $100\times$ scale, MSR’s read-latency penalty drops from $1.77\times$ to $1.01\times$. At cloud scales, MSR’s small-storage advantage over MBR is therefore essentially free in latency.

1 Introduction

Production storage systems at exabyte scale cannot afford full replication: the storage tax is too high, and large clusters see daily disk failures. Facebook’s HDFS-RAID deployment, for example, replaced 3-way replication with a $(10, 4)$ Reed–Solomon code to recover petabytes of capacity, and then a custom locally-repairable code to cut the resulting repair traffic [6]. Erasure codes, from RAID-6 and Reed–Solomon to recent *regenerating* codes such as MSR and MBR [1, 4], trade replication for parity computation. They promise the same

fault tolerance as multi-way replication using only a fraction of the storage, and — in the case of regenerating codes — a fraction of the repair-network bandwidth as well.

Yet the regenerating-code half of this design space is almost untouched in practice. Mainstream production-grade erasure-coding libraries — Jerasure, Intel ISA-L, HDFS-EC, Ceph — ship Reed–Solomon and locally-repairable variants, but none ships an MSR or MBR implementation. The academic literature mirrors that gap: existing work on regenerating codes is overwhelmingly construction- and bound-oriented, with very little side-by-side empirical comparison against the RAID/RS family that operators actually deploy. A practitioner deciding whether the storage savings of MSR or the repair-bandwidth savings of MBR are worth the engineering cost has essentially no measured answer to fall back on.

What is much less well understood, then, is what these codes look like under *live load* and how they sit relative to the mature RAID/RS family. Most prior work compares codes analytically or with microbenchmarks. The question I set out to answer is concrete: when a disk fails and the system must serve foreground I/O in degraded mode, which code wins on tail latency, where do the regenerating codes actually land in the storage–bandwidth–latency tradeoff space against RAID and RS, and how does all of that change as the workload, the dataset, or the failure scenario varies?

This paper makes three contributions. (1) A unified C++ implementation of seven codes (RAID-0, RAID-1, RAID-6, RAID-10, RS, MSR, MBR) sharing one abstract interface — including from-scratch MSR and MBR implementations, which to my knowledge are not provided by any production-grade erasure-coding library — with the four most representative evaluated end-to-end on the same workload, simulator and failure scenario. (2) A trace-driven discrete-event simulator built around a *DiskRequest DAG* abstraction whose `on_ready` closures let each code express its $GF(2^8)$ compute inline with its I/O dependencies, without leaking coding logic into the simulator. (3) An empirical placement of MSR and MBR in the storage–bandwidth–latency tradeoff space alongside the mature RAID/RS codes, including the headline

finding that the regenerating-code degraded-mode penalty is a fixed α -symbol cost that *amortises* as the dataset grows: at $100\times$ scale MSR’s penalty all but disappears. The rest of the paper covers background and the threat model (§2), system design (§3), implementation details (§4), experiments (§5), discussion (§6), and conclusion (§7).

2 Background and Motivation

Erasure codes. An (n, k) erasure code maps k data symbols onto n stored symbols on n distinct nodes. A code is *maximum-distance-separable* (MDS) if any k of the n nodes suffice to reconstruct the original data; classical Reed–Solomon [3, 5] and RAID-6 are MDS. *Regenerating* codes [1] relax the per-node read pattern: instead of pulling k entire nodes during repair, a new node contacts d *helper* nodes and downloads only β symbols from each, for a total repair traffic of $d\beta$. Each node stores α symbols per stripe.

The storage–repair tradeoff at fixed (n, k) . For fixed (n, k, d) , (α, β) trace out a curve. Two endpoints are of practical interest: the *minimum-storage regenerating* (MSR) point minimises $\alpha = d - k + 1$, and the *minimum-bandwidth regenerating* (MBR) point minimises $d\beta$ at the cost of $\alpha = d$. For $(n, k, d) = (6, 3, 5)$ this gives MSR $\alpha = 3$ and MBR $\alpha = 5$. The two families are summarised below:

Family	Storage	Repair BW	Examples
MDS	optimal	k symbols	RS, RAID-6
Regenerating	near-opt.	$d\beta$ symbols	MSR, MBR

Reliability is a separate axis. The four codes I evaluate — RAID-10(6, 3), RS(6, 3), MSR(6, 3, 5) and MBR(6, 3, 5) — have *non-identical* reliability profiles even at the same (n, k) . RS, MSR and MBR all tolerate any $n - k = 3$ disk failures. RAID-10(6, 3), in contrast, tolerates 3 simultaneous failures only if no two of them are mirrored siblings; otherwise the array loses data. The four points therefore lie in a tradeoff *space*, not on a single Pareto frontier; reliability is held only partially fixed by the parameter choice. I return to this caveat when discussing the storage–repair plot in §5.

The implementation and evaluation gap. Reed–Solomon and RAID-6 are mature: production libraries (Jerasure, Intel ISA-L, HDFS-EC, Ceph) ship them, and decades of deployment experience characterise their behaviour. Regenerating codes are not. None of the mainstream libraries above ships an MSR or MBR implementation, and the published literature on these codes remains weighted toward existence proofs, capacity bounds, and code constructions rather than empirical comparisons against the RAID/RS family they aim to displace. The result is a design space with one half mapped in detail and the other half mapped only on paper. This project

sim_main	/	test suite
DESSimulator		StorageArray
		TraceTranslator
RAID-0/1/6/10, RS, MSR, MBR		
gf256, matrix_gf256		

Figure 1: Two-layer architecture. The codes library has no dependency on the simulator; both executors consume the same `DiskRequest` DAG.

contributes implementations of MSR and MBR alongside RAID-10 and RS under one interface specifically so the two halves can be compared head-to-head.

Why latency under load is the right question. Real workloads are skewed in size, op-mix, and arrival distribution. Public Microsoft Research Cambridge (MSRC) block I/O traces [2] from production servers see request sizes from 0.5 KB to 250 KB and read/write ratios that vary by an order of magnitude across volumes. Closed-form analyses of repair bandwidth do not predict how a code will saturate, what its tail latency looks like in degraded mode, or how either changes with dataset size — exactly the questions a deployment must answer.

Threat model. Steady-state replay of a real workload, single permanent disk failure mid-trace, foreground I/O continues in degraded mode. I do *not* overlap repair traffic with foreground I/O in the timing experiments; repair correctness is verified separately. Multi-failure and repair-under-load are future work (§6).

3 Design

The system is two layers (Fig. 1). A **codes library** implements each erasure code behind one abstract `ErasureCode` interface; a **simulator library** replays traces against either an in-memory `StorageArray` (for correctness verification) or a discrete-event simulator `DESSimulator` (for timing). The two layers communicate through a single data structure — the `DiskRequest` — that carries enough metadata to express read–compute–write dependencies without ever coupling coding math to scheduling.

The `ErasureCode` interface. Three calls dominate: `translate(lb, nb, rw, data, bs)` converts a logical I/O into a vector of `DiskRequests`; `assembleRead(...)` re-assembles logical data from the filled read requests; and `repair(disk_cap, bs)` returns the DAG that rebuilds a failed disk. The same calls drive both modes: `block_size = 0` means *sim mode* (no data movement, timing-only), `> 0` means *verify mode* (data buffers carried). Codes generate requests; they never touch storage.

The DiskRequest DAG. This is the system’s central abstraction. Each request carries an `id`, a `depends_on` list of local IDs that must complete first, an optional shared buffer `buf`, and an optional `on_ready` closure invoked just before the request is issued. The closure’s job is to do the $GF(2^8)$ work that sits *between* the dependent reads and the dependent write — for example, in a parity update: read old data and old parity, XOR-and-multiply to compute the new parity, then write it. Without `on_ready`, the simulator would have to know about parity arithmetic; with it, the simulator only walks the DAG. The verifier executes the DAG via Kahn’s topological sort; the simulator decrements per-request `dep_counts` on each `DiskCompletion` event and enqueues newly-ready requests.

Two-phase experiment timeline. Each experiment replays the trace twice against an isolated array: once with no failure (*normal* phase, baseline) and once with a designated disk marked failed before replay starts (*degraded* phase). The two phases share their seeded arrival sequence so latency differences come purely from the code’s degraded-mode I/O pattern. Repair latency is not measured under load; the repair DAG is exercised separately for correctness (§4).

4 Implementation

$GF(2^8)$ math. `gf256.{h,cpp}` provides `log/antilog` tables for addition (XOR), multiplication, inverse and exponentiation, all $O(1)$ table lookups. `matrix_gf256.{h,cpp}` layers Gauss–Jordan inversion, matrix multiply, and Cauchy/Vandermonde generators on top.

Per-code constructions. **RAID-0/1/10** are striping and mirroring (no GF math); only RAID-10 is exercised in the experiments. **RAID-6** computes a P (XOR) and a Q ($\sum_i \alpha^i x_i$) parity column over $GF(2^8)$; single-disk repair is XOR, double-disk repair inverts a 2×2 GF system. **RS**(n, k) uses a systematic Cauchy generator; repair of $f \leq n - k$ failed disks inverts the corresponding $k \times k$ submatrix. **MSR**(n, k, d) follows the layered RS construction with $\alpha = d - k + 1$ symbols per node. **MBR**(n, k, d) uses the Rashmi–Shah–Kumar product-matrix construction [4] with $\alpha = d$. Each code lives in one `.h/.cpp` pair under `src/codes/`.

Discrete-event simulator. `DESSimulator` is a priority queue keyed on event time. Events are `RequestArrival`, `DiskCompletion`, and `DiskFailure`. Each disk has its own FIFO queue; service time is deterministic `size/bandwidth`. When a request lands, its `dep_count` (the number of unfinished prerequisites in its DAG) is checked; if zero, `on_ready` fires and the request is enqueued. A `DiskCompletion` decrements every dependent’s count and may release new requests cascading through the DAG.

Trace translator. `TraceTranslator` reads pre-processed MSRC rows of the form (`op`, `offset`, `size`), discards the original timestamps, and re-emits them as *Poisson* arrivals at a configurable rate. This decouples workload *shape* (which I want to keep realistic) from arrival *rate* (which I sweep).

Verification by construction. Each code is exercised in a separate test (`test_raid`, `test_ec`) against an in-memory `StorageArray`: encode-then-decode round-trip, single- and double-disk failure followed by `repair()`, read-back of the rebuilt disk, and partial-stripe (RMW) writes at unaligned boundaries. The contract is byte-for-byte equality. All seven codes pass. Repair latency is not part of the timing sweep in §5; this section is where repair is exercised.

5 Experiments

5.1 Setup

Parameter	Value
Disks n	6
(k, d)	(3, 5)
Codes evaluated	RAID-10, RS, MSR, MBR
Disk bandwidth	128 MB/s
Block / chunk	4 KB / 1 block
Trace	MSRC <code>mds_1</code> (2007)
Requests per phase	10 000
Phases	normal, degraded
Arrivals	Poisson, rate-swept
Failure	disk 0 (data); disk 3 (parity) for RS
RNG seed	42

I run two dataset sizes, `mds_1` and `mds_1_100x`; the latter scales the addressable dataset $100\times$ by spreading the same trace across a proportionally larger logical address space, so that each stripe is touched by many more logical I/Os.

5.2 The storage–repair tradeoff

Figure 2 plots the four codes in the (repair-download, total-storage) plane, normalised to RAID-10. The MSR and MBR points sit on the analytic MSR $\leftrightarrow$ MBR boundary curve; RS lies far to the right (it pays k symbols of repair traffic per failed symbol); RAID-10 sits at (1, 1) by construction. As cautioned in §2, the four points are *not* comparable as a strict Pareto frontier: RAID-10 trades reliability for those low coordinates.

5.3 Load–latency: normal vs. degraded

Figure 3 sweeps Poisson arrival rate against average latency for both dataset sizes. At low load all codes converge near the disk service time. As load rises the curves bend upward at different points; the saturation knee is where the response cliff begins.

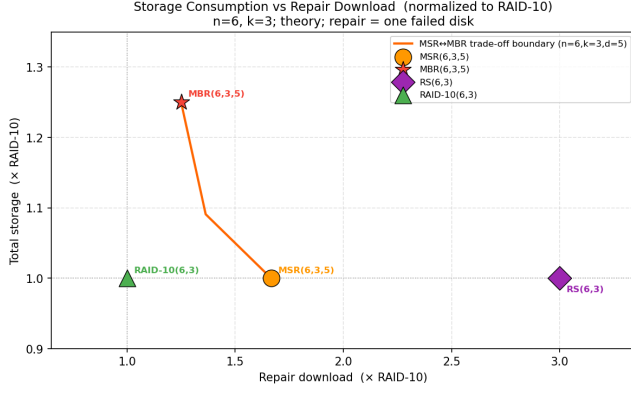


Figure 2: Storage vs. repair download, normalised to RAID-10. The four evaluated codes do *not* share the same reliability guarantee: RAID-10(6, 3) tolerates 3 failures only if no siblings fail, whereas RS/MSR/MBR tolerate any 3.

1× dataset (Fig. 3, left): the saturation order is $MBR < MSR < RS < RAID-10$. MBR saturates first because its $\alpha = d = 5$ makes every logical I/O fan out to many disk requests; RAID-10 saturates last because each I/O hits at most a single stripe replica.

100× dataset (Fig. 3, right): the order becomes $MBR < MSR \approx RS \approx RAID-10$. MSR closes the gap to the MDS codes once its α cost is amortised across many more logical I/Os per stripe; MBR still trails because its α is larger.

In both panels the degraded curves (dashed) peel away from the normal curves (solid) at mid-load; the gap is much smaller at 100×.

5.4 Degraded-mode latency penalty

Figure 4 reports the ratio of degraded-phase to baseline latency at very low load (so queueing effects do not mask the coding cost). At 1×, regenerating codes pay heavily: MSR’s read penalty is 1.77×, MBR’s 2.77×, while RS adds only 1.11× on a data disk and effectively zero on a parity disk; RAID-10 is at the baseline. Writes show the same ordering and amplified ratios (RAID-10 1.00×, RS 1.37×, MSR 2.70×, MBR 4.29×) because each write touches more stripes worth of repair work.

5.5 Effect of dataset size: α amortisation

The 100× panel of Fig. 4 is the headline result. At cloud-storage scale the regenerating-code penalty all but vanishes: MSR’s read penalty drops from 1.77× to 1.01× and its write penalty from 2.70× to 1.03×. MBR’s penalty also shrinks (2.77× → 1.26× on reads, 4.29× → 1.29× on writes), but it is asymptotically larger because $\alpha = d$ is bigger. The reason is structural (§6): each repair pays a fixed cost *per α -symbol*

stripe, and a 100× dataset spreads that fixed cost across 100× as many logical I/Os.

6 Discussion

Why α -amortisation works. Repair (and degraded-mode reads, which structurally resemble repair) pays a fixed cost per α -symbol stripe: read α surviving symbols, do $GF(2^8)$ work, write α recovered symbols. That cost is independent of how many logical I/Os hit the stripe. Larger datasets dilute it across more I/Os, so the per-request penalty falls.

Practical implication. At cloud scale the storage cost *dominates* the choice between MSR and MBR: MBR pays 1.25× more storage for an asymptotic latency disadvantage. MSR’s 1% read penalty at 100× is essentially free relative to its storage savings.

Limitations. I model a single permanent failure; multi-failure correlated scenarios are not exercised. Disk service time is deterministic, ignoring internal queueing in real devices. $GF(2^8)$ compute time is treated as instantaneous, which understates CPU-bound codes. Most importantly, repair traffic is not overlapped with foreground I/O in the latency sweep — a real system would have to amortise the α -cost *plus* the cost of the rebuild itself.

7 Conclusion

I implemented four representative erasure codes — including from-scratch MSR and MBR, which are absent from production erasure-coding libraries — under one abstract interface and drove them with a trace-replay DES whose central DiskRequest-DAG abstraction lets each code express its $GF(2^8)$ work as closures attached to its I/O dependencies. Putting the regenerating codes side-by-side with the mature RAID/RS family on the same workload places them concretely in the storage–bandwidth–latency tradeoff space rather than only on paper: the textbook storage–repair tradeoff at fixed (n, k) holds in my measurements, and the regenerating-code degraded-mode penalty amortises with dataset size to $\sim 1\%$ for MSR at 100× scale — a practical answer to the question of whether MSR is deployable that the prior literature did not provide. Future work: multi-failure scenarios, repair traffic concurrent with foreground I/O, $GF(2^8)$ compute-time accounting, and lazy-vs-eager repair scheduling.

References

- [1] Alexandros G. Dimakis, P. Brighten Godfrey, Yunnan Wu, Martin J. Wainwright, and Kannan Ramchandran. Network coding for distributed storage systems. *IEEE*

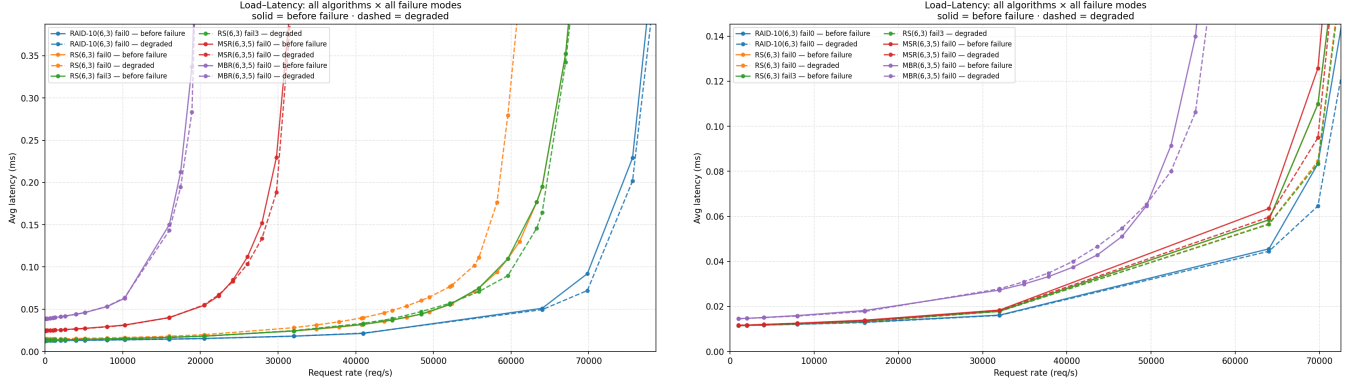


Figure 3: Load-latency at $1\times$ (left) and $100\times$ (right) dataset scale. Solid lines are normal phase, dashed lines are degraded. Saturation order at $1\times$: $MBR < MSR < RS < RAID-10$; at $100\times$ the gap between MSR and the MDS codes closes.

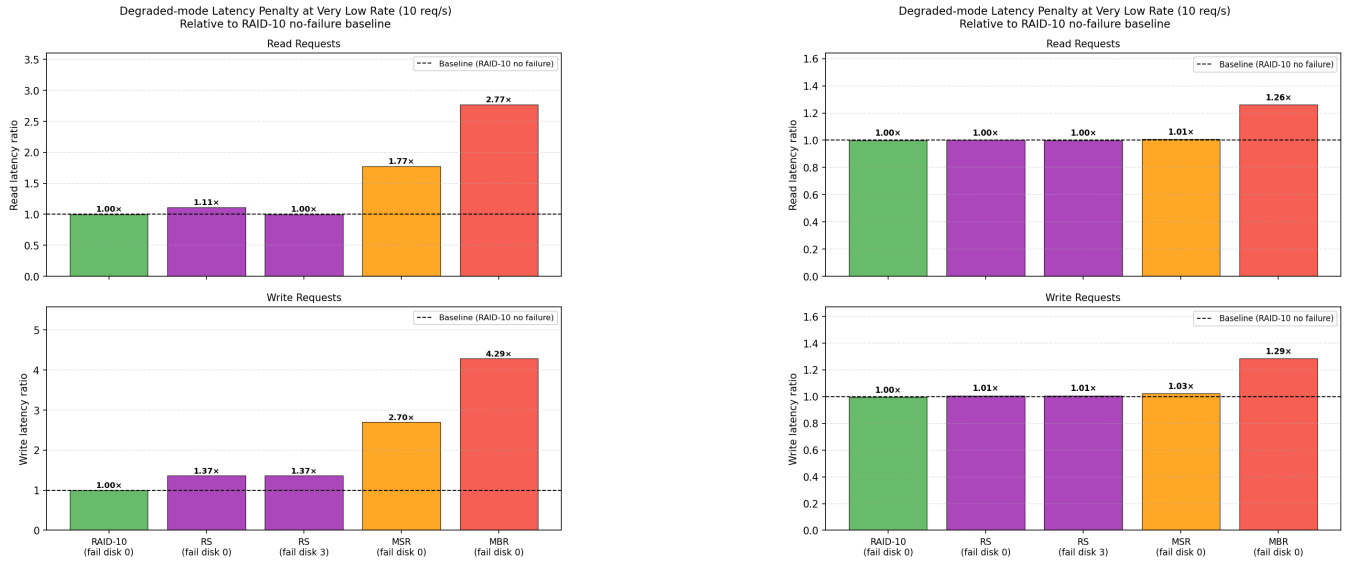


Figure 4: Degraded-mode latency penalty (relative to RAID-10 no-failure baseline) at $1\times$ scale (left) and $100\times$ scale (right). At $100\times$ MSR drops to $1.01\times$ on reads; the regenerating-code penalty amortises with dataset size.

Transactions on Information Theory, 56(9):4539–4551, 2010.

- [2] Dushyanth Narayanan, Austin Donnelly, and Antony Rowstron. Write off-loading: Practical power management for enterprise storage. In *USENIX Conference on File and Storage Technologies (FAST)*, 2008.
- [3] James S. Plank. A tutorial on Reed–Solomon coding for fault-tolerance in RAID-like systems. *Software: Practice and Experience*, 27(9):995–1012, 1997.
- [4] K. V. Rashmi, Nihar B. Shah, and P. Vijay Kumar. Optimal exact-regenerating codes for distributed storage at the MSR and MBR points via a product-matrix construction. *IEEE Transactions on Information Theory*, 57(8):5227–5239, 2011.

- [5] Irving S. Reed and Gustave Solomon. Polynomial codes over certain finite fields. *Journal of the Society for Industrial and Applied Mathematics*, 8(2):300–304, 1960.
- [6] Maheswaran Sathiamoorthy, Megasthenis Asteris, Dimitris Papailiopoulos, Alexandros G. Dimakis, Ramkumar Vadali, Scott Chen, and Dhruba Borthakur. XORing elephants: Novel erasure codes for big data. *Proceedings of the VLDB Endowment*, 6(5):325–336, 2013.